1. This is a **sample project**, very small in size but written and managed just like the other (graded) course projects. While it is small, the code you write for it will be needed in several other projects (LUTHOR, NFAMATCH, ZOBOS and CZAR) so it is far from "busy work".
2. Here is the change log for this assignment write-up. I will try to be descriptive in my log messages.
3. You can also subscribe to this page and receive Emails when changes are made.

In this assignment will you will familiarize yourself with the course's grading scripts and setting up a suitable development environment for your projects.

1. Your first step is the latter, read about your options in the course DockerFile page, follow the appropriate instructions for the development environment of your preference.
2. The second and last step is to complete this sample project for the course.

The name of your application for this project **must be** `ALPHABETENCODING` (case sensitive).

Your core task is to write **encoding** and **decoding** functions for one of the character escaping mechanisms we'll use in the course. Ideally, you will do this in the your language of choice for the course projects.

Alphabet Encoding Definition

Characters **NOT** in the ranges `'!'` through `'9'`, `';'` through `'['`, `']'` through `'w'` and `'y'` through `'~'` (notice the missing `':'`, `'\'` and `'x'`) are **always** written in a hexadecimal escape sequence: `xHH` where `H` is an upper or lower case hexadecimal value.

In C and C++ a character value `c` can be tested for **required encoding** with `!(c!='x' && c!='\\' && c!=':' && isprint(c) && !isspace(c))`, where `isprint()` and `isspace()` are standard routines in "libc"; many languages have equivalent library routines.

Other characters may be written in escaped form (`xHH`) **or** as their visual glyph (for instance `A` for ASCII decimal code 65).

We will call this "Alphabet Encoding."

Input

Your `ALPHABETENCODING` app will run in either **encode** or **decode** mode, each has a specific sequence of command line arguments.

I/O errors

In all invocations, inability to create, access, read or write to the file argument provided on the command line should result in no `OUPUT` lines **and** a non-zero exit status from your `ALPHABETENCODING` application.

decode

Argument	Value
1	the case-sensitive word `decode`
2	A single token (no whitespace) of encoded data
3	The output path name to store the decoded result, a pre-existing file should be overwritten

Here are two examples of `decode` use:

```
$ ./ALPHABETENCODING decode hellox20x63x72x75x65x6cx20worldx0a output.txt
$ cat output.txt
hello cruel world
$
```

```
$ ./ALPHABETENCODING decode x5cooox5cx20x5cooox5c result.txt
$ cat result.txt
\ooo\ \ooo\$
```

Notice that in the last example, the encoded argument **didn't end** with `x0a`, a newline (`\n`, `std::endl`). So `result.txt`'s last character is a `\`, and we see the shell prompt (`$`) displayed immediately after the file contents. (YMMV, this is what happens on the author's machine. Some `cat(1)` program may take care to push the `/dev/tty` device before they exit.)

encode

Argument	Value
1	the case-sensitive word `encode`
2	The input file path name containing the data to encode

Your encoded result should be printed to the `stdout` (`std::cout`) device according to the OUTPUT formatting requirements. You might want to also see the examples here.

Here is an example of `encode` use, a near inverse of a previous decode example:

```
$ cat output.txt
```

```
hello cruel world
$ ./ALPHABETENCODING encode output.txt
OUTPUT :hellox20cruelx20worldx0a:
```

Why doesn't the encoded result match the encoded argument of the previous decode example? Because character values that **aren't required** to be `xHH` encoded, may still be encoded. The decode example had all the letters of "cruel" encoded. The encode example did not. **Both encodings are correct.**

```
$ ./ALPHABETENCODING decode x5cooox5cx20x5cooox5c result.txt
$ cat result.txt
\ooo\ \ooo\$
```

Other Requirements

Provide a "README" in the `doc` directory per submission requirements. In this, state:

1. What type of development environment have you adopted (of the three choices in DockerImage: docker, `COMPGRADING.tar.bz2` on your own Linux box, or "other").
2. If not "other", how much time did it take to get your development environment setup? Please estimate in hours.
3. How long did it take to complete this ALPHABETENCODING project? Please estimate in hours.

grader.sh

In this course, students are provided the same tarball the grader will use for testing their project. While this project is graded as a participation point, you want to practice using a `grader.sh` script and get your questions and concerns answered in the next lecture.

First, set up a development environment according to this page. If you don't use the docker image, be sure `COMPGRADING` is correctly specified in your environment.

Second, download this tarball to your development environment and unroll it in a temporary directory.

```
$ ls alphabetencoding-student.*
alphabetencoding-student.tar.bz2
$ mkdir ~/tmp
$ cd ~/tmp
$ tar xjf ../alphabetencoding-student.tar.bz2
:
$ ls
alphabetencoding/
```

Now go to the directory holding your `ALPHABETENCODING` and execute the `grader.sh` script from the `alphabetencoding-student.tar.bz2` resource.

```
$ cd ~/compilers/a-to-z-encoding
$ ls ALPHABETENCODING
```

```
ALPHABETENCODING
$ ~/tmp/alphabetencoding/grader.sh
:
:
:
```

This script runs several encode and decode tests on your `ALPHABETENCODING` program.

Like all other `grader.sh` scripts in the course, you have to hit `ENTER` and continue **ONLY** when a failing test has occurred. When this happens, you should read the screen carefully; there are typically important clues, if not actual debugging hints, displayed for your benefit.

If the `grader.sh` terminates with summary results and you haven't hit `ENTER`, then all the tests passed. Woot!

Checking and Submitting Your Work

Here are some things to double check your submission against:

i. Your archive does not raise errors when "unrolling" with the `${COMPGRADING}/explode-subm` script.

```
$ echo $COMPGRADING
/COMPGRADING
$ ls
my-project.zip
$ "${COMPGRADING}/explode-subm" -P PROJECT my-project.zip ./here
```

ii. `PROJECT` is different for each project, the first two (for example) are `ALPHABETENCODING` and `LUTHOR`.
iii. Your archive contains only the essential files, **don't provide unneeded files**
iv. Don't provide files that have been provided to you (such as test cases in the `*-student.tar.bz2` files); that would just be silly.
v. Make sure the `grader.sh` script for the assignment runs to completion.

> You aren't required to use `.zip` files, `explode-subm` knows about the usual suspects: `.tar`, `.tar.gz`, `.tar.bz2` ...

When you are happy with your work, submit your project archive file for grading at https://www.gradescope.com.

Rubric

Every graded project has a detailed rubric, typically with several `grader.sh` tests per line item. This project will be "graded" as a participation point, but it has an example rubric so you are familiar with what other projects will look like. This work would be worth 26 programming assignment points.

Requirements	Points	Notes
Meets compilers course project requirements	10	
`README` (three items)	4	
Meets I/O error requirements (exit status 1 on inaccessible files or empty file input)	4	
`decode` operations	4	
`encode` operations	4	

Assignments/AlphabetEncoding (last edited 2026-01-22 13:19:34 by khellman)